# Distributed Computing Seminar

## Lecture 6: Distributed Filesystems

# Outline

- Filesystems Overview
- NFS (Network File System)
- GFS (Google File System)

# Filesystems Overview

- System that permanently stores data
- Usually layered on top of a lower-level physical storage medium
- Divided into logical units called “files”
  - Addressable by a *filename* (“foo.txt”)
  - Usually supports hierarchical nesting (directories)
- A file *path* joins file & directory names into a **relative** or **absolute** address to identify a file (“/home/aaron/foo.txt”)

# Distributed Filesystems

- Support access to files on remote servers
- Must support concurrency
  - Make varying guarantees about locking, who “wins” with concurrent writes, etc...
  - Must gracefully handle dropped connections
- Can offer support for replication and local caching
- Different implementations sit in different places on complexity/feature scale

# NFS

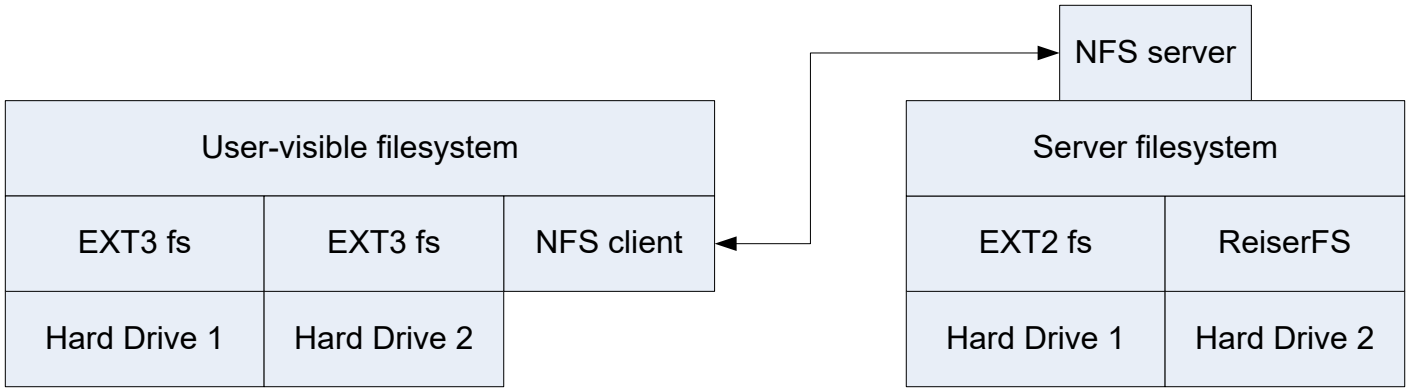
- First developed in 1980s by Sun
- Presented with standard UNIX FS interface
- Network drives are *mounted* into local directory hierarchy

# NFS Protocol

- Initially completely stateless
  - Operated over UDP; did not use TCP streams
  - File locking, etc., implemented in higher-level protocols
- Modern implementations use TCP/IP & stateful protocols

# Server-side Implementation

- NFS defines a *virtual file system*
  - Does not actually manage local disk layout on server
- Server instantiates NFS volume on top of local file system
  - Local hard drives managed by concrete file systems (EXT, ReiserFS, ...)
  - Other networked FS's mounted in by...?



# NFS Locking

- NFS v4 supports stateful locking of files
  - Clients inform server of intent to lock
  - Server can notify clients of outstanding lock requests
  - Locking is lease-based: clients must continually renew locks before a timeout
  - Loss of contact with server abandons locks

# NFS Client Caching

- NFS Clients are allowed to cache copies of remote files for subsequent accesses
- Supports *close-to-open* cache consistency
  - When client A closes a file, its contents are synchronized with the master, and timestamp is changed
  - When client B opens the file, it checks that local timestamp agrees with server timestamp. If not, it discards local copy.
  - Concurrent reader/writers must use flags to disable caching

# NFS: Tradeoffs

- NFS Volume managed by single server
  - Higher load on central server
  - Simplifies coherency protocols
- Full POSIX system means it “drops in” very easily, but isn’t “great” for any specific need

# The Google/Hadoop File System

Sanjay Ghemawat, Howard Gobioff,
and Shun-Tak Leung

(These slides by Alex Moshchuk, University of Washington)

# Motivation

- Google needed a good distributed file system
  - Redundant storage of massive amounts of data on cheap and unreliable computers
- Why not use an existing file system?
  - Google's problems are different from anyone else's
    - Different workload and design priorities
  - GFS/HDFS is designed for Google apps and workloads
  - Google/Cloud apps are designed for GFS/HDFS

# DFS

Distributed File Systems & GFS

Don't move data to workers… move workers to the data!

- Store data on the local disks of nodes in the cluster
- Start up the workers on the node that has the data local

Why?

- Not enough RAM to hold all the data in memory
- Disk access is slow, but disk throughput is reasonable

A distributed file system is the answer

- GFS (Google File System) for Google's MapReduce
- HDFS (Hadoop Distributed File System) for Hadoop

# Assumptions - HDFS

- High component failure rates
  - Inexpensive commodity components fail all the time
- “Modest” number of HUGE files
  - Just a few million
  - Each is 100MB or larger; multi-GB files typical
- Files are write-once, mostly appended to
  - Perhaps concurrently
- Large streaming reads
- High sustained throughput favored over low latency

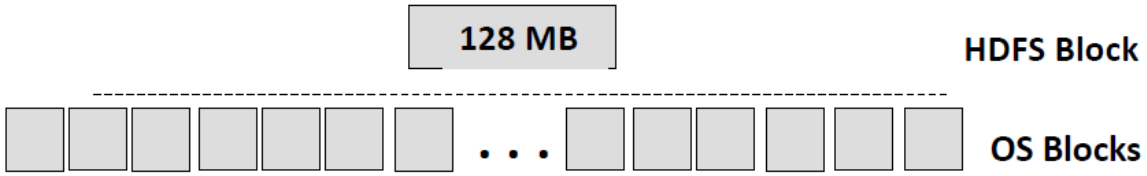
# HDFS Files

- User data divided into 64/128MB blocks and replicated across local disks of cluster node to address:
  - Cluster network bottleneck
  - Cluster node crashes
- Master/Slave Architecture
  - Master (Namenode) maintains a name space and metadata
  - Slaves (Datanodes): maintain (three/X) copies of each data block

# HDFS - Blocks

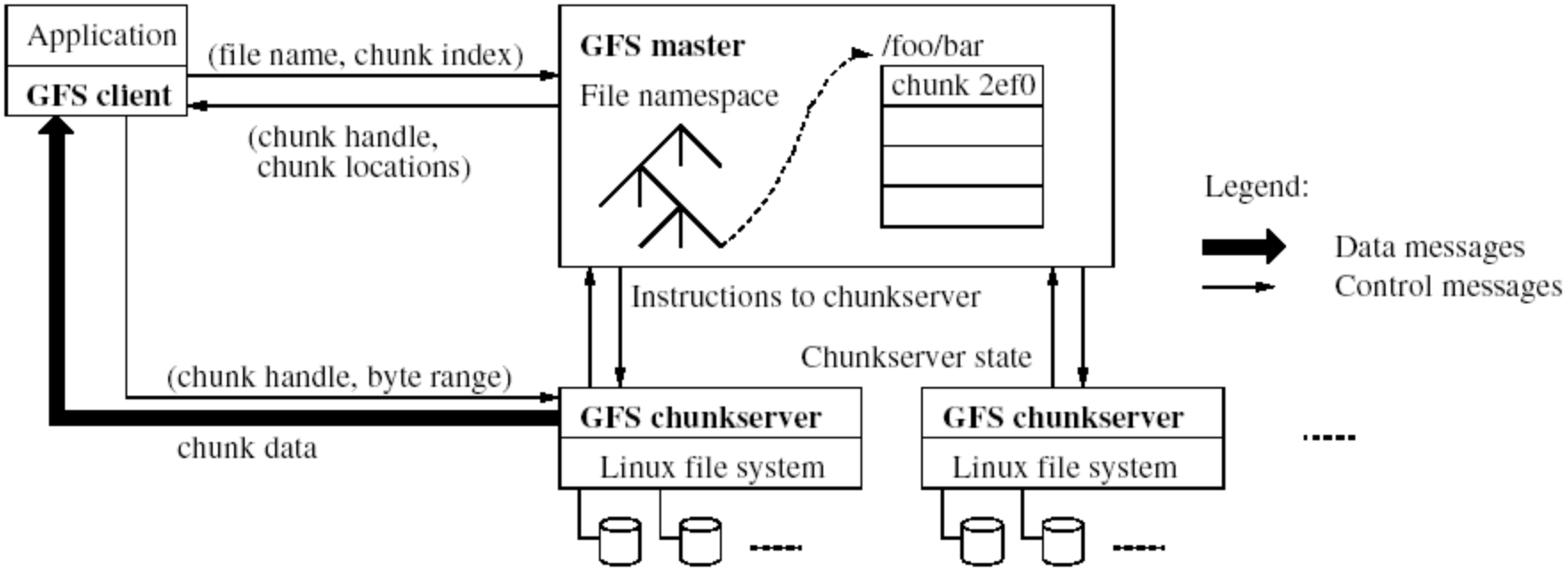
- Hadoop file system that runs on top of existing file system
- File Blocks128MB compared to 4K OS blocks



# HDFS Architecture

- Single master
- Mutiple chunkservers/Datanodes




*…Can anyone see a potential weakness in this design?*

# Namenode

- Arbitrator and repository for all HDFS metadata
- Data never flows through Namenode
- Executes file system namespace operations
  - open, close, rename files and directories
- Determines mapping of blocks to Datanodes

# Metadata (1/2)

- Global metadata is stored on the master
  - File and chunk namespaces
  - Mapping from files to chunks/blocks
  - Locations of each chunk's replicas
- All in memory (64 bytes / chunk)
  - Fast
  - Easily accessible

# Metadata (2/2)

- Master has an *operation log* for persistent logging of critical metadata updates
  - persistent on local disk
  - replicated
  - checkpoints for faster recovery

# EditLog & FsImage

- Managed by Namenode
  - Stored in files on the local OS file system
- EditLog
  - Transaction log
  - Records all changes to file system metadata
- FsImage
  - Image of entire file system namespace
  - Mappings of blocks to files
  - File system properties
  - Stored in a file on the local OS file system

# Datanodes

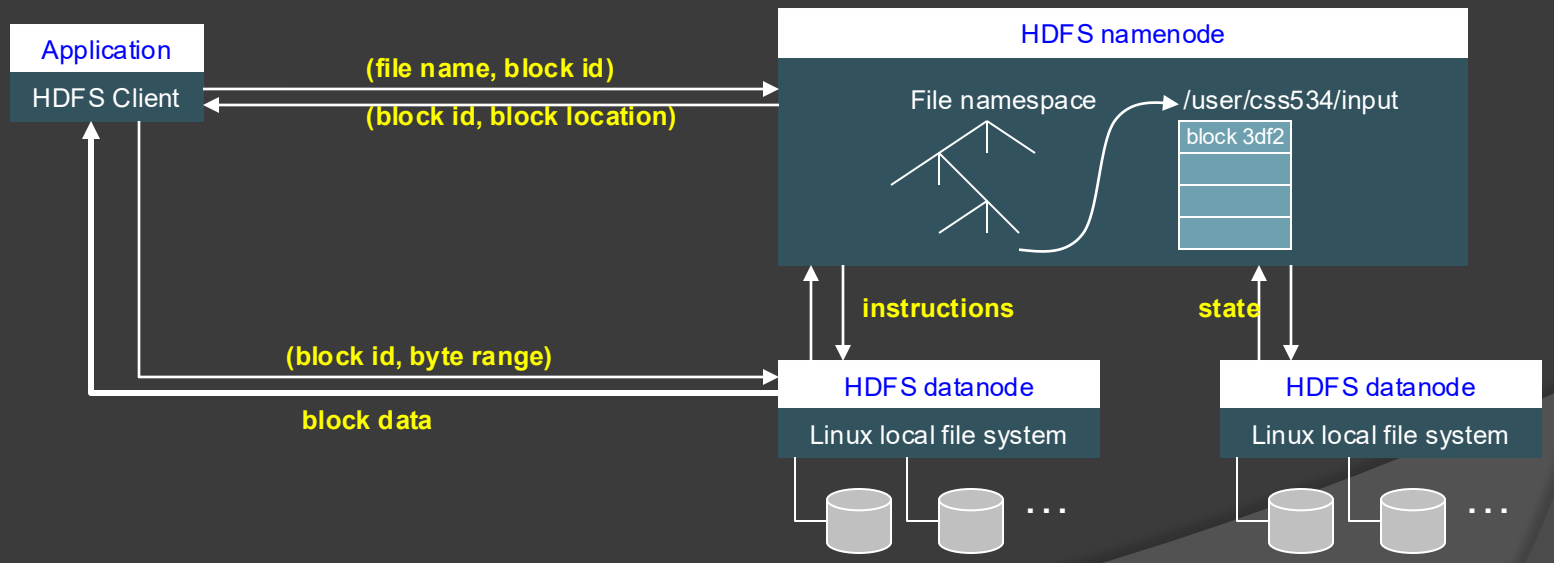
- Serve read / write requests from client
- Block creation, deletion and replication upon instruction from Namenode
- No knowledge of HDFS files
- Stores HDFS data in files on local file system
  - Determines optimal file count per directory
  - Creates subdirectories automatically

# Single master

- From distributed systems we know this is a:
  - Single point of failure
  - Scalability bottleneck
- HDFS solutions:
  - Shadow masters
  - Minimize master involvement
    - never move data through it, use only for metadata
      - and cache metadata at clients
    - large chunk size
    - master delegates authority to primary replicas in data mutations (chunk leases)
- Simple, and good enough!

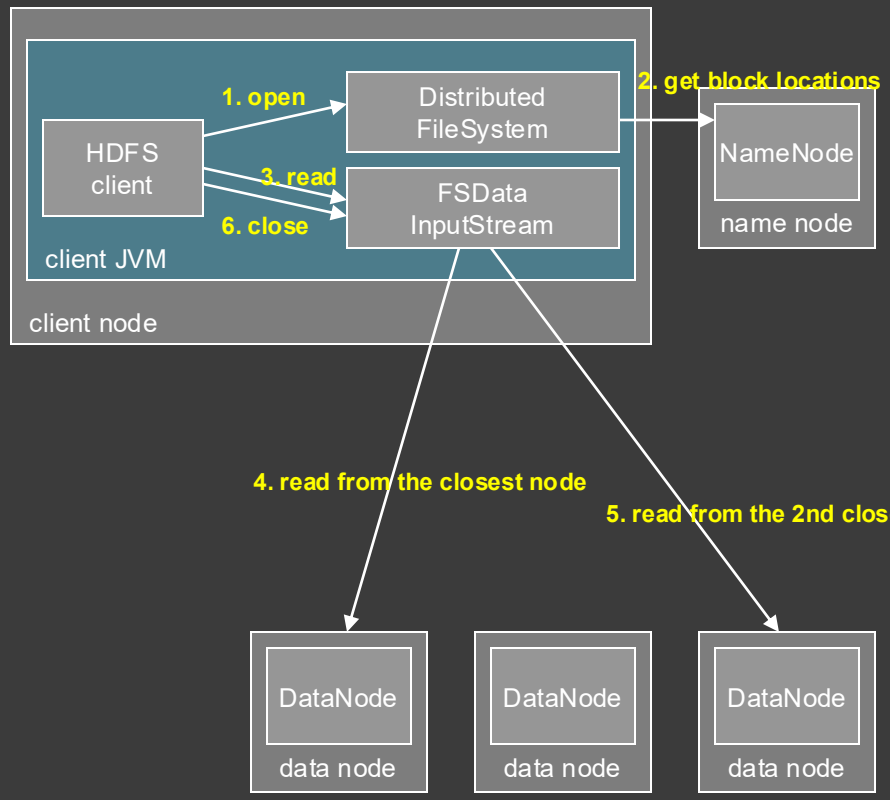
# HDFS: Hadoop Distributed File Systems

- Client requests meta data about a file from namenode
- Data is served directly from datanode

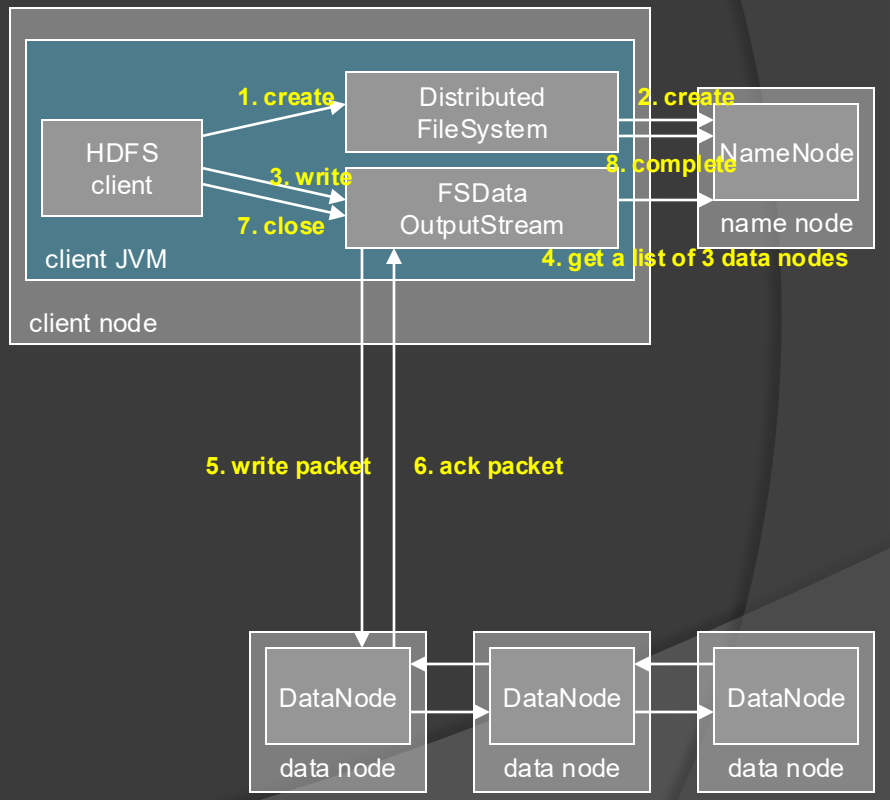


# File Read/Write in HDFS

- File Read

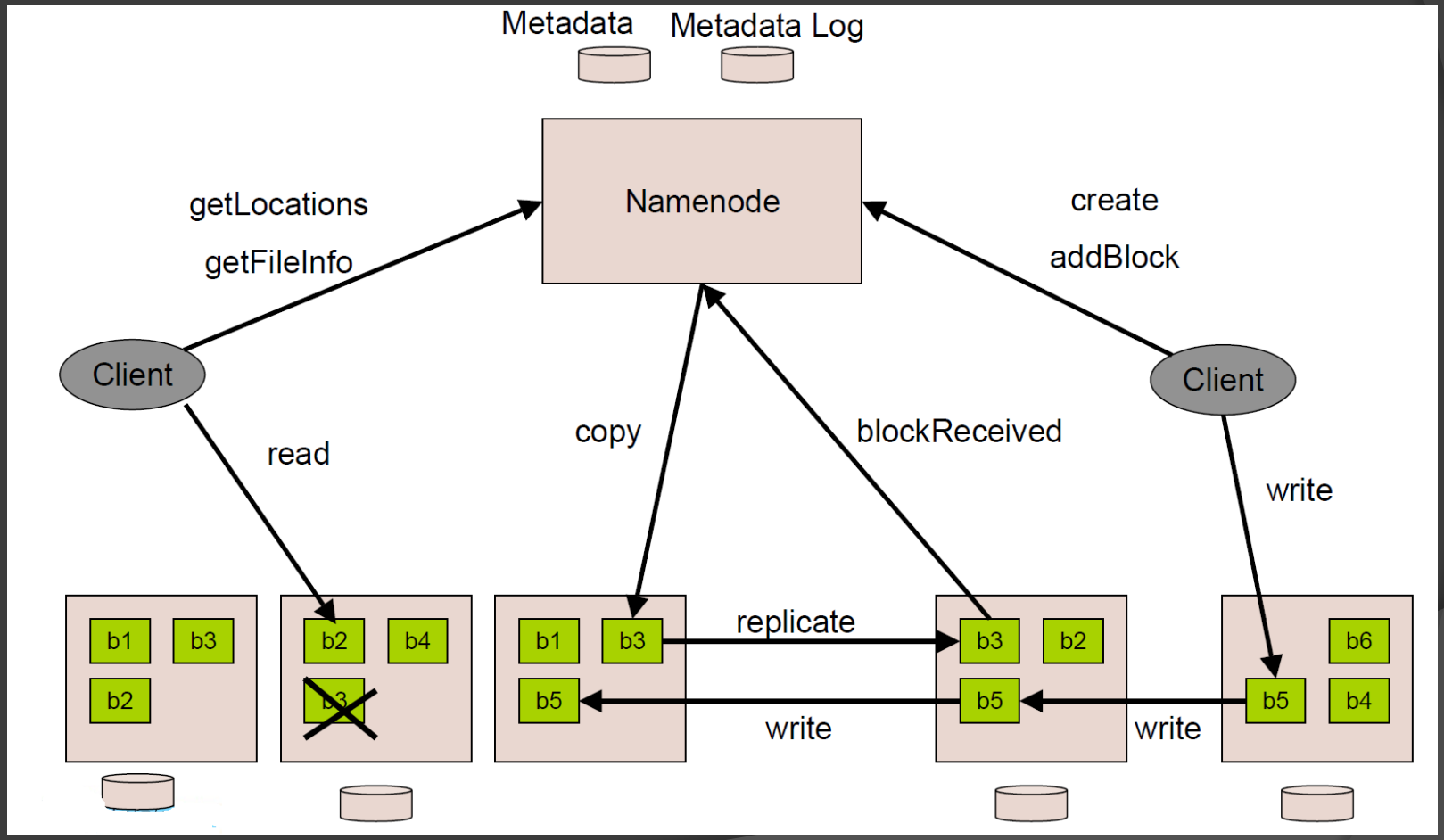



- File Write




If a data node crashed, the crashed node is removed, current block receives a newer id so as to delete the partial data from the crashed node later, and Namenode allocates an another node.

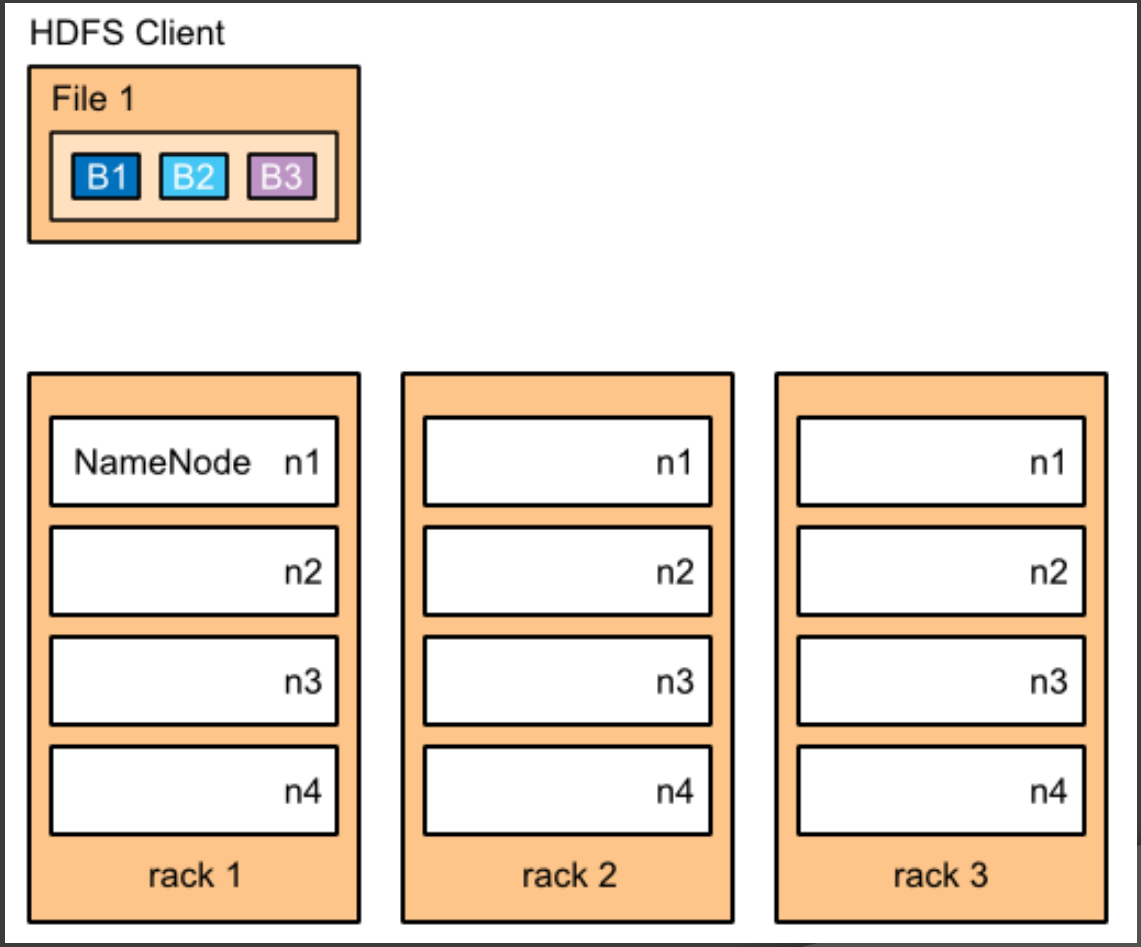
# HDFS operations



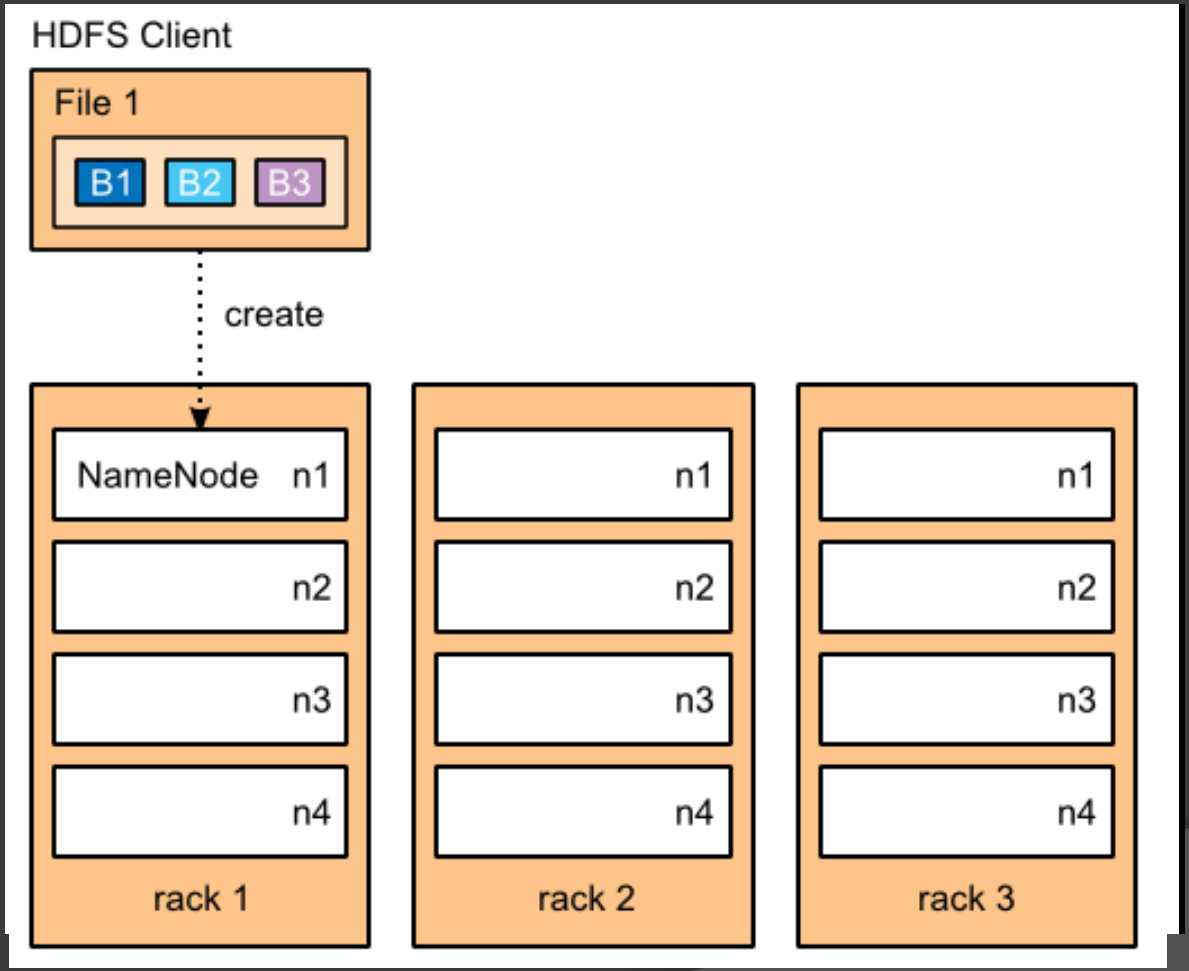
# Replica Placement

- Distinguishes HDFS from most other DFS
- When replication factor == 3
  - Put one replica on local rack
  - Put one replica on different node on local rack
  - Put one replica on different node on different rack
- Replicas do not evenly distribute across racks

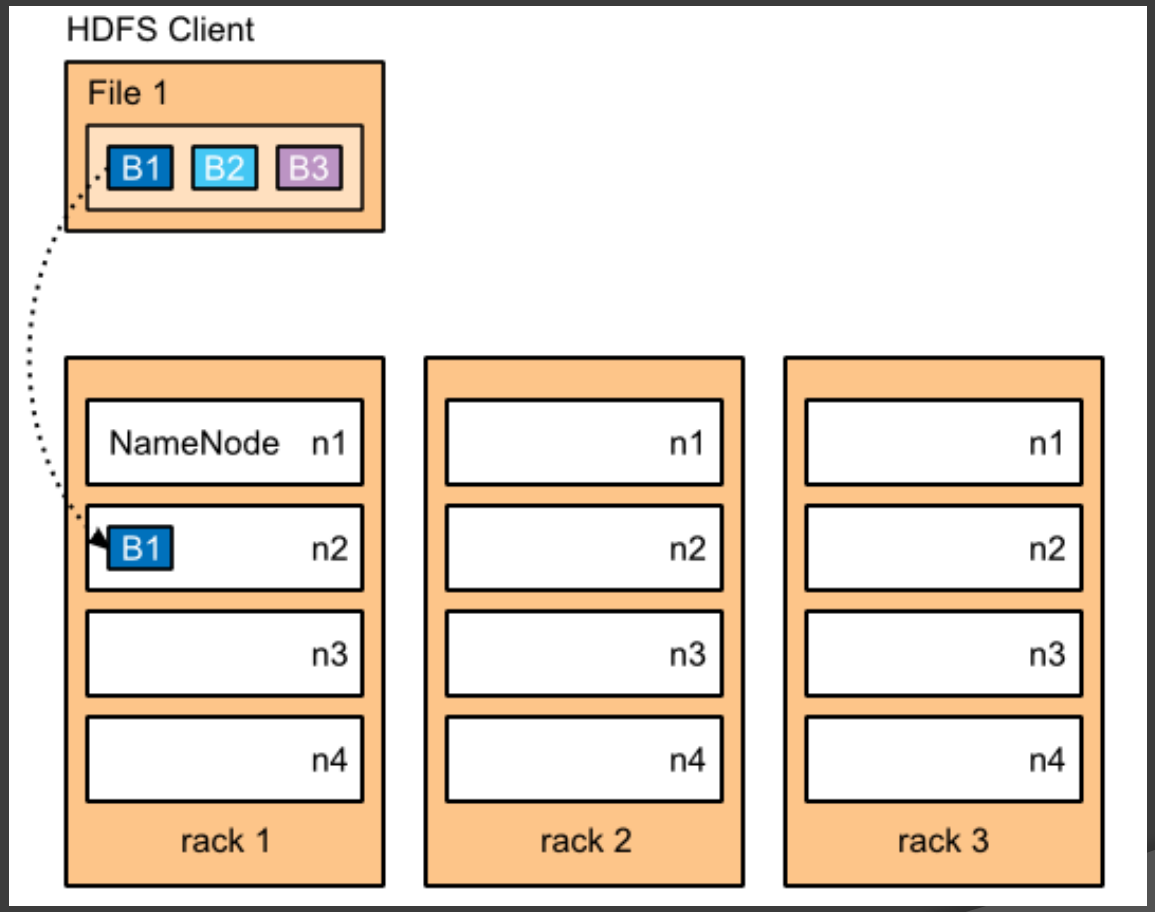
# File Write


HDFS Client
File 1
B1
B2
B3
NameNode
n1
n2
n3
n4
rack 1
n1
n2
n3
n4
rack 2
n1
n2
n3
n4
rack 3

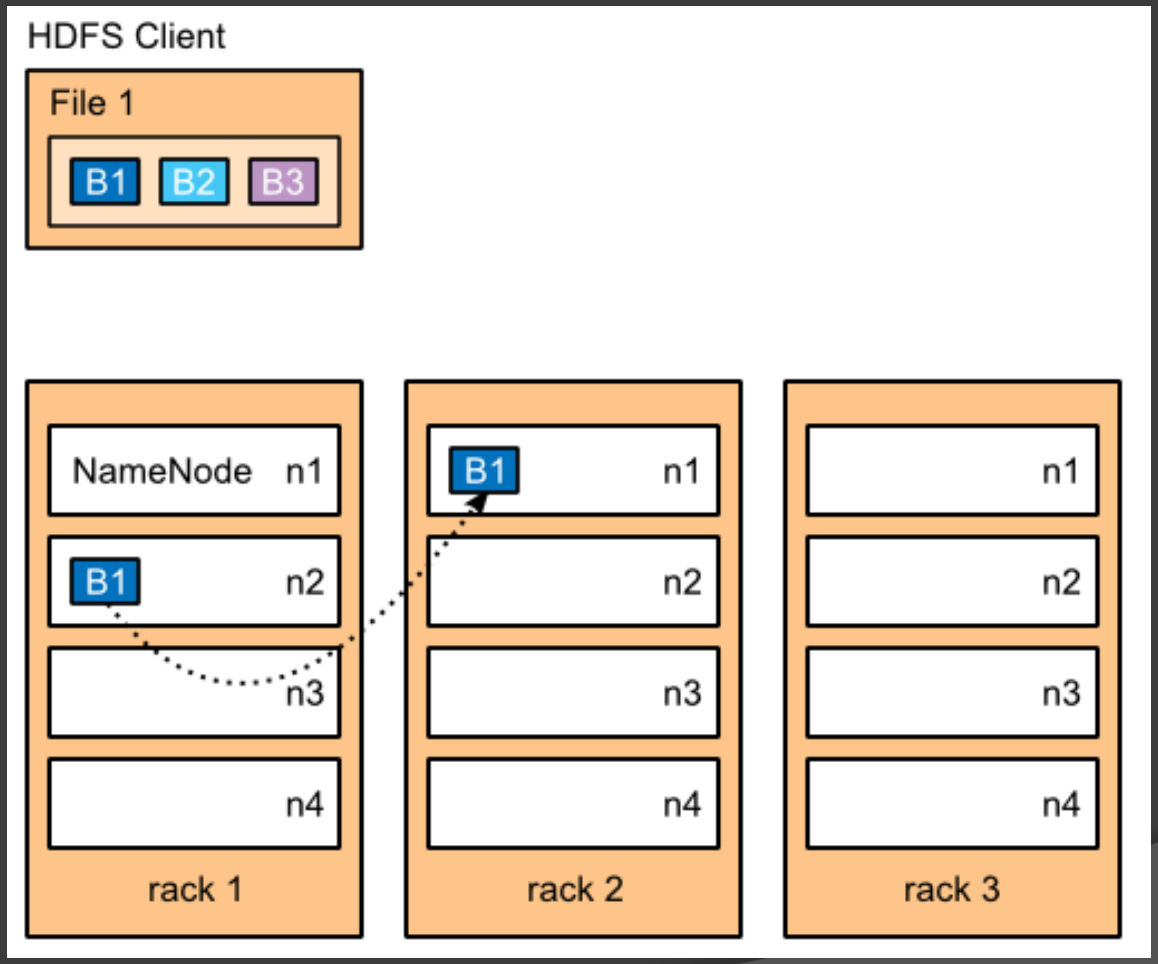
# File Write


HDFS Client
File 1
B1
B2
B3
create
NameNode
n1
n2
n3
n4
rack 1
n1
n2
n3
n4
rack 2
n1
n2
n3
n4
rack 3

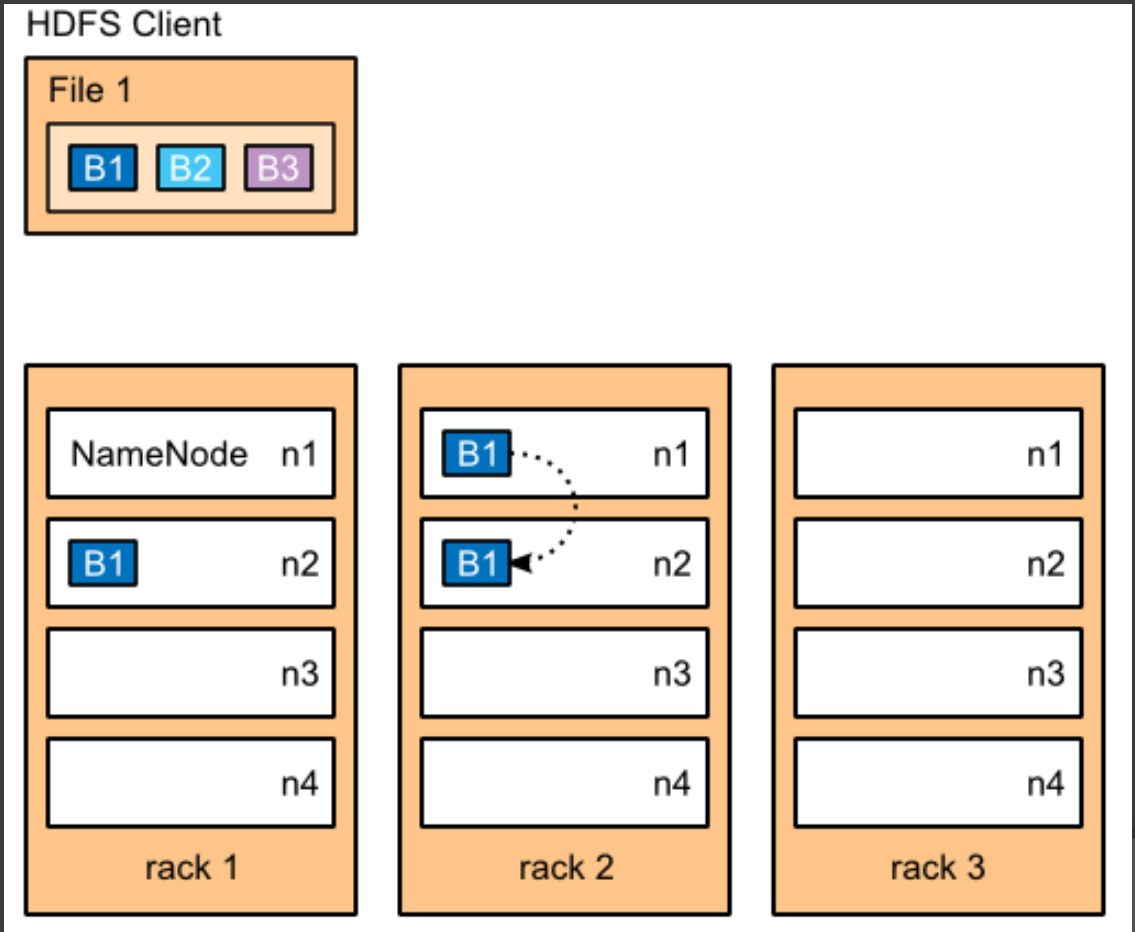
# File Write


HDFS Client
File 1
B1
B2
B3
NameNode
n1
B1
n2
n3
n4
rack 1
n1
n2
n3
n4
rack 2
n1
n2
n3
n4
rack 3

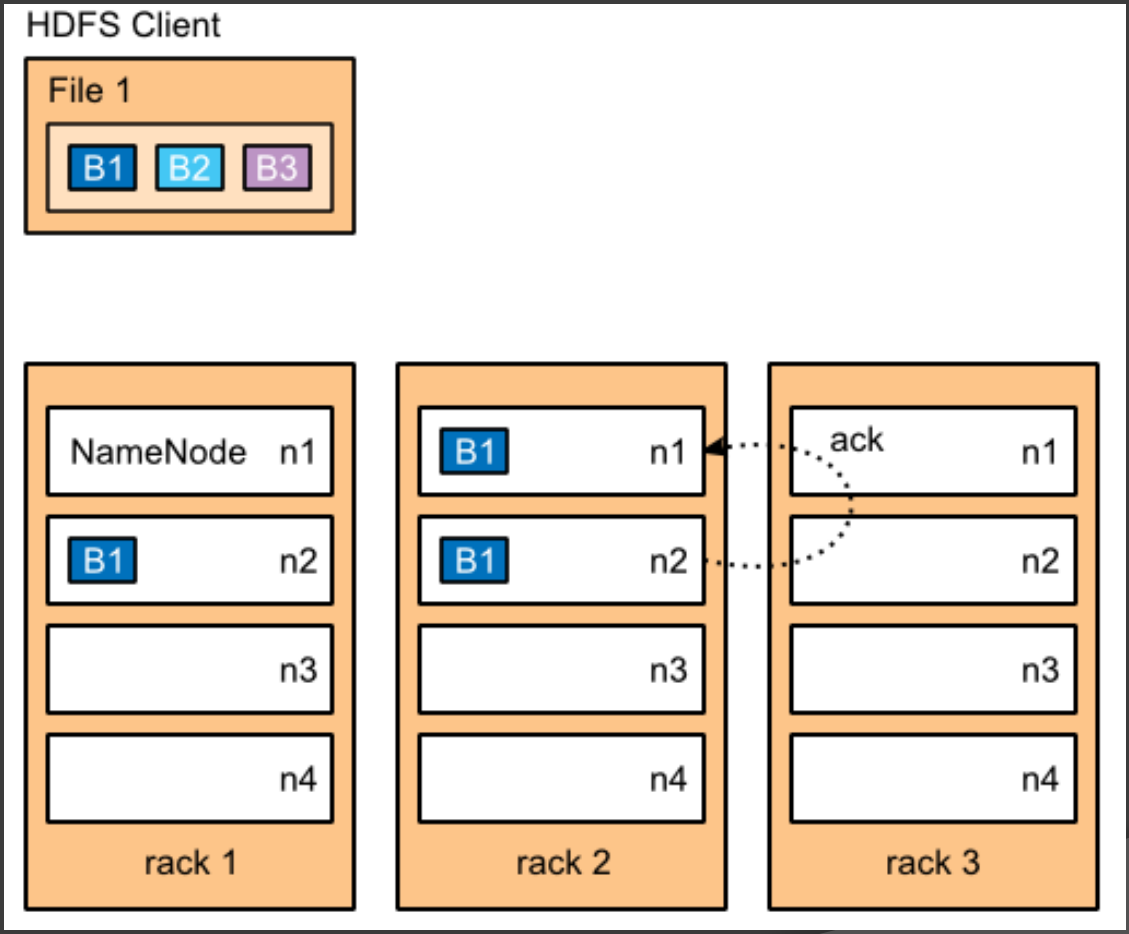
# File Write


HDFS Client
File 1
B1
B2
B3
NameNode
n1
B1
n2
n3
n4
rack 1
B1
n1
n2
n3
n4
rack 2
n1
n2
n3
n4
rack 3

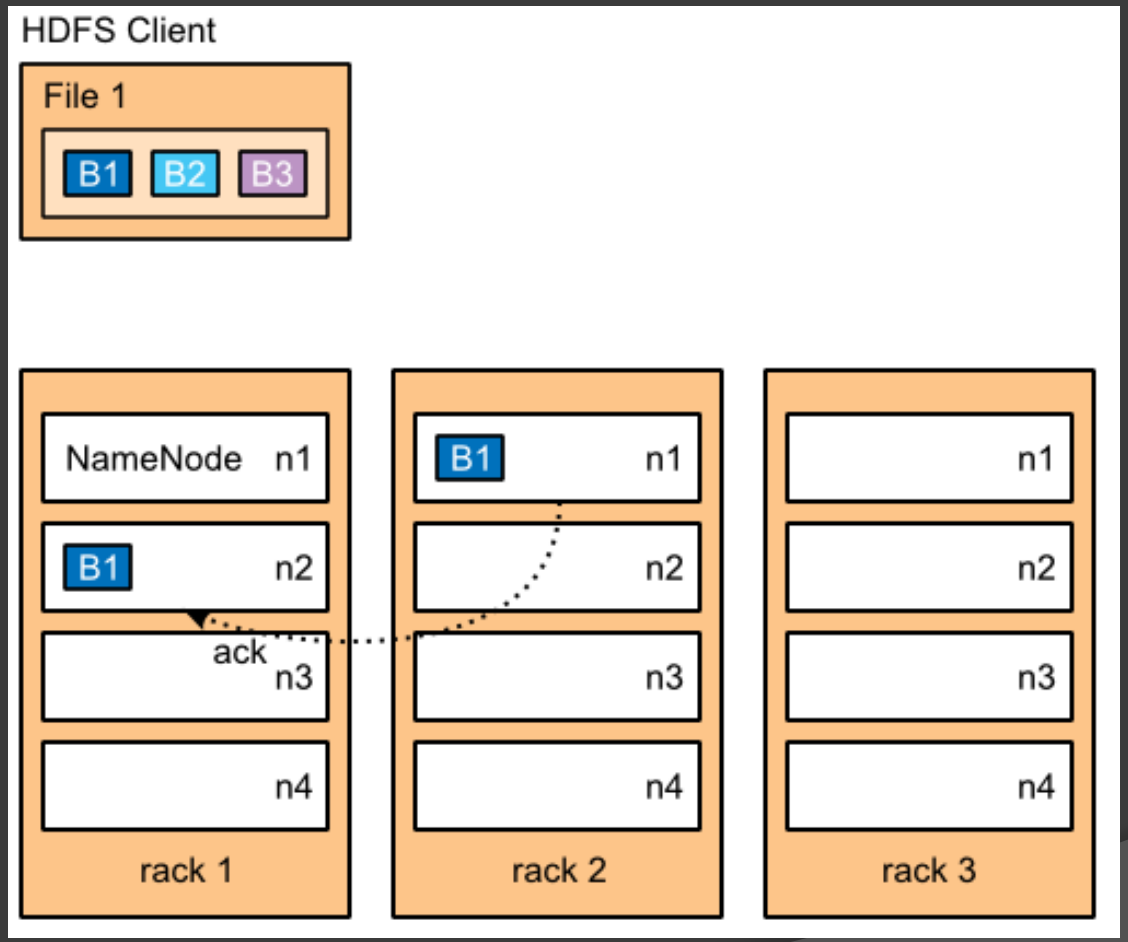
# File Write


HDFS Client
File 1
B1
B2
B3
NameNode
n1
B1
n2
n3
n4
rack 1
B1
n1
B1
n2
n3
n4
rack 2
n1
n2
n3
n4
rack 3

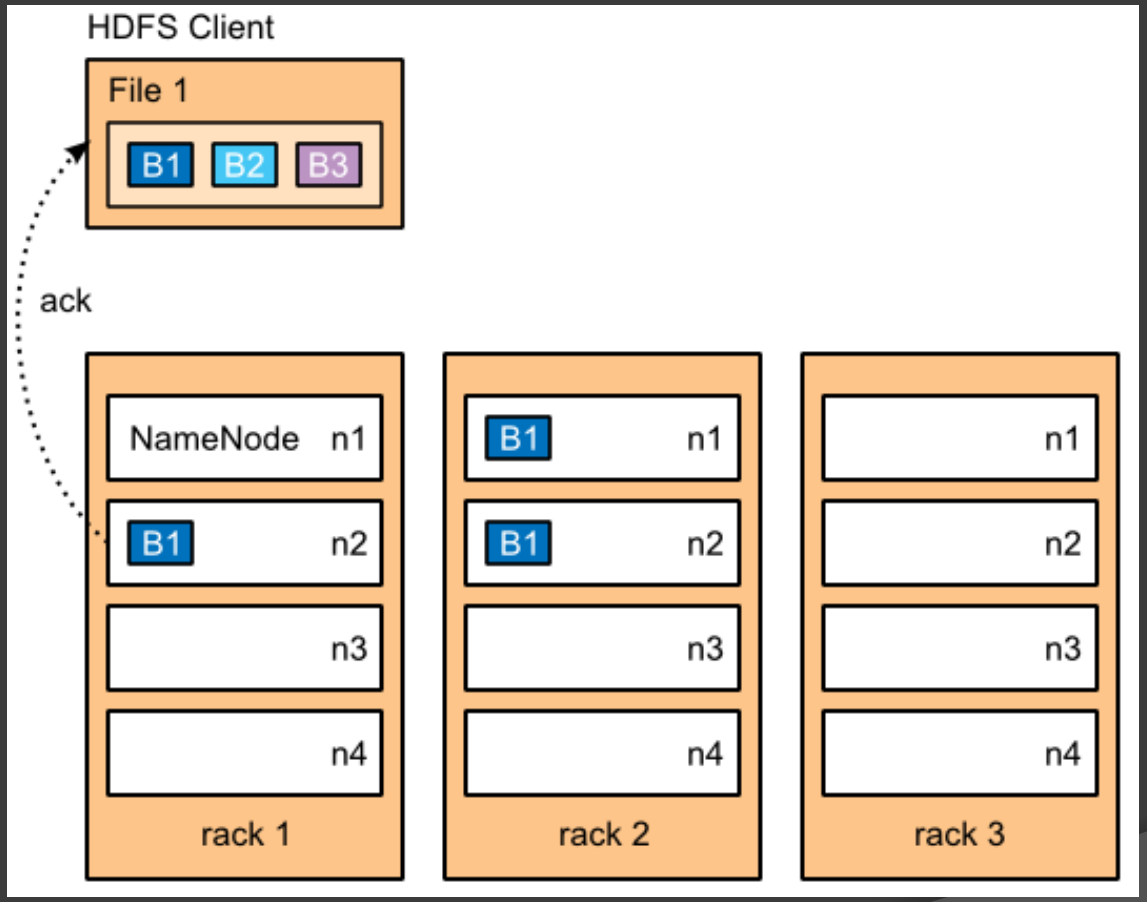
# File Write


HDFS Client
File 1
B1
B2
B3
NameNode n1
B1 n2
n3
n4
rack 1
B1 n1
B1 n2
n3
n4
rack 2
ack n1
n2
n3
n4
rack 3

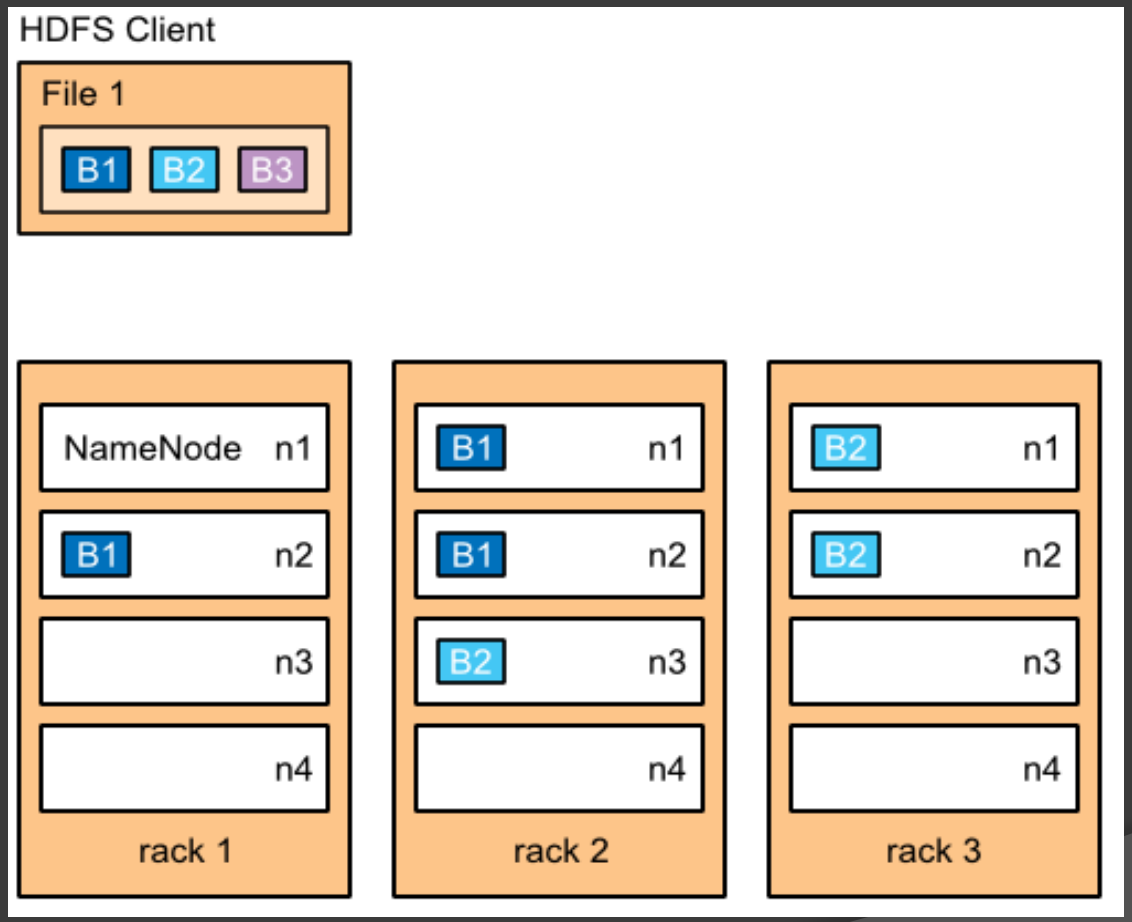
# File Write


HDFS Client
File 1
B1
B2
B3
NameNode
n1
B1
n2
ack
n3
n4
rack 1
B1
n1
n2
n3
n4
rack 2
n1
n2
n3
n4
rack 3

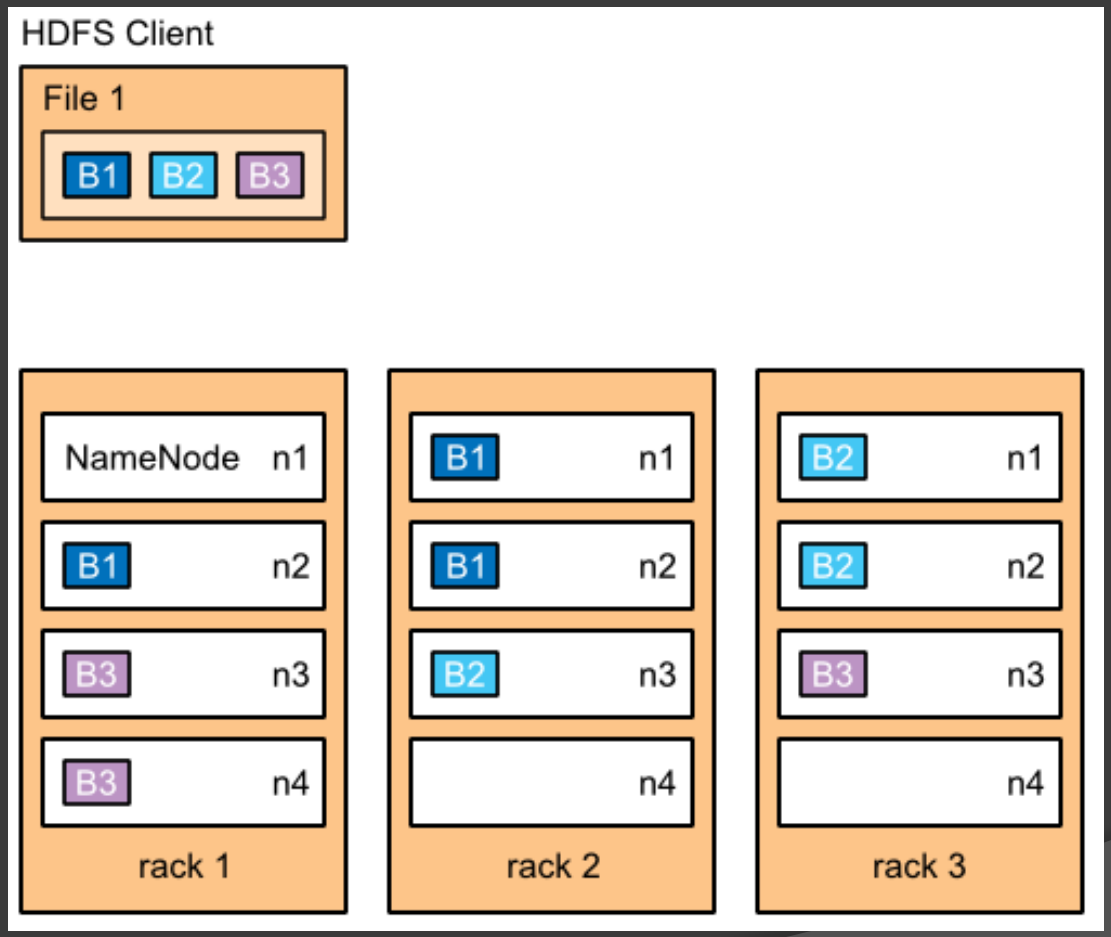
# File Write


HDFS Client
File 1
B1
B2
B3
ack
NameNode
n1
B1
n2
n3
n4
rack 1
B1
n1
B1
n2
n3
n4
rack 2
n1
n2
n3
n4
rack 3

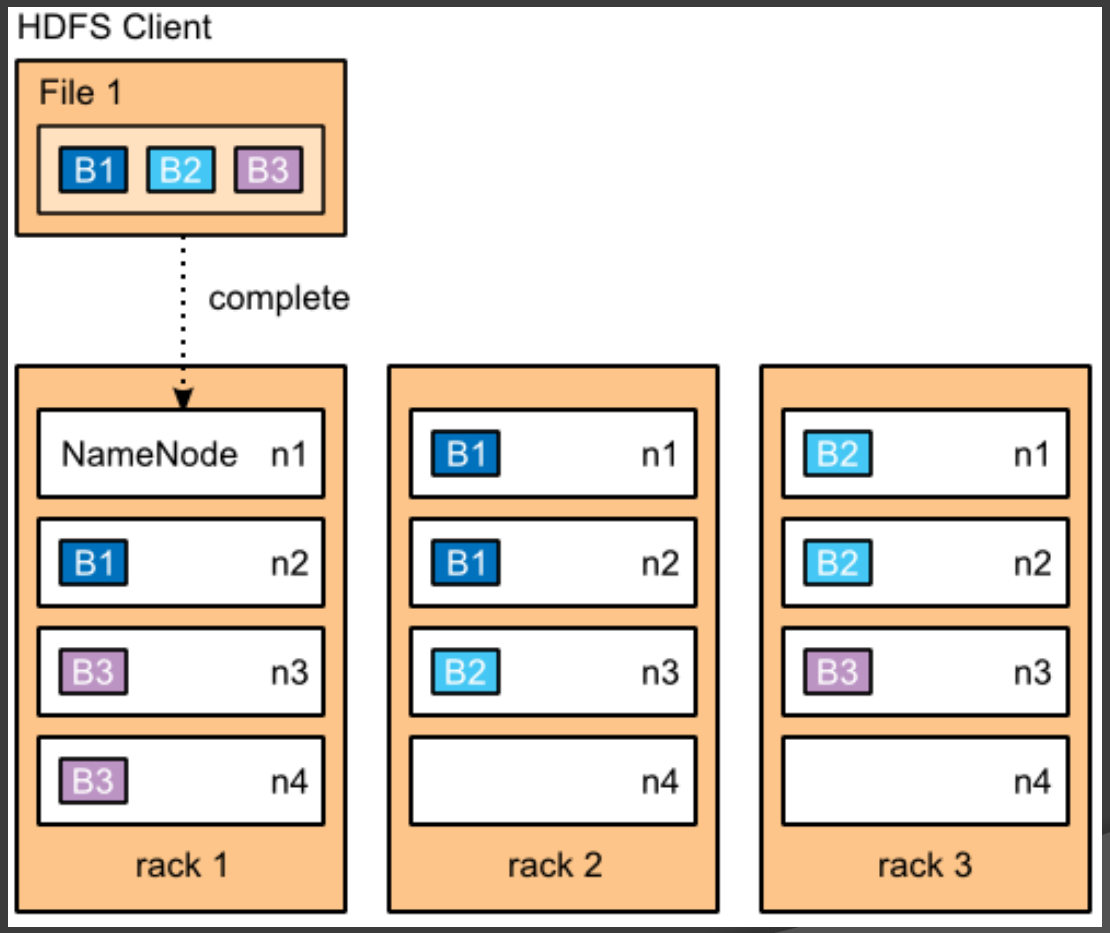
# File Write


HDFS Client
File 1
B1
B2
B3
NameNode
n1
B1
n2
n3
n4
rack 1
B1
n1
B1
n2
B2
n3
n4
rack 2
B2
n1
B2
n2
n3
n4
rack 3

# File Write


HDFS Client
File 1
B1
B2
B3
NameNode
n1
B1
n2
B3
n3
B3
n4
rack 1
B1
n1
B1
n2
B2
n3
n4
rack 2
B2
n1
B2
n2
B3
n3
n4
rack 3

# File Write

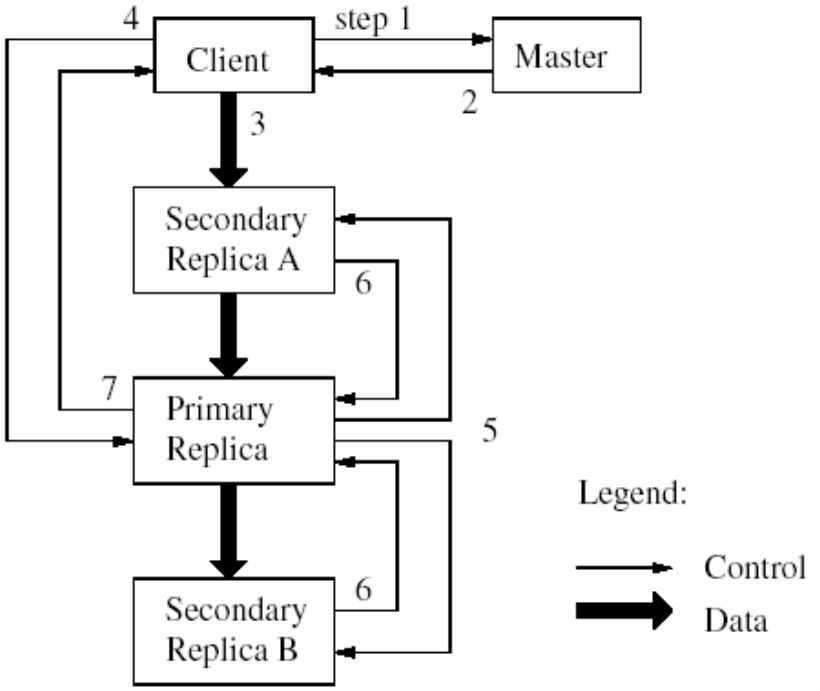

HDFS Client
File 1
B1
B2
B3
complete
NameNode
n1
B1
n2
B3
n3
B3
n4
rack 1
B1
n1
B1
n2
B2
n3
n4
rack 2
B2
n1
B2
n2
B3
n3
n4
rack 3

# Start-Up Process

- Namenode enters Safemode
  - Replication does not occur in Safemode
- Each Datanode sends Heartbeat
- Each Datanode sends Blockreport
  - Lists all HDFS data blocks
- Namenode creates Blockmap from Blockreports
- Namenode exits Safemode
- Replicate any under-replicated blocks

# Mutations

- Mutation = write or append
  - must be done for all replicas
- Goal: minimize master involvement
- Lease mechanism:
  - master picks one replica as primary; gives it a “lease” for mutations
  - primary defines a serial order of mutations
  - all replicas follow this order
- Data flow decoupled from control flow



# Atomic record append

- Client specifies data
- HDFS appends it to the file atomically at least once
  - GFS picks the offset
  - works for concurrent writers
- Used heavily by Google apps
  - e.g., for files that serve as multiple-producer/single-consumer queues

# Master's responsibilities (1/2)

- Metadata storage
- Namespace management/locking
- Periodic communication with chunkservers
  - give instructions, collect state, track cluster health
- Chunk creation, re-replication, rebalancing
  - balance space utilization and access speed
  - spread replicas across racks to reduce correlated failures
  - re-replicate data if redundancy falls below threshold
  - rebalance data to smooth out storage and request load

# Master's responsibilities (2/2)

- Garbage Collection
  - simpler, more reliable than traditional file delete
  - master logs the deletion, renames the file to a hidden name
  - lazily garbage collects hidden files
- Stale replica deletion
  - detect "stale" replicas using chunk version numbers

# Fault Tolerance

- High availability
  - fast recovery
    - master and chunkservers restartable in a few seconds
  - chunk replication
    - default: 3 replicas.
  - shadow masters
- Data integrity
  - checksum every 64KB block in each chunk

# Limitations

- Write-once model
  - Plan to support appending-writes =>OK
- A namespace with an extremely large number of files exceeds Namenode's capacity to maintain
- Cannot be mounted by exisiting OS
  - Getting data in and out is tedious
  - Virtual File System can solve problem

# Limitations

HDFS does not implement / support
- User quotas
- Access permissions
- Hard or soft links
- Data balancing schemes

No periodic checkpoints

Namenode is single point of failure
- Automatic restart and failover to another machine not yet supported

# Performance

| Cluster | A | B |
|---|---|---|
| Chunkservers | 342 | 227 |
| Available disk space | 72 TB | 180 TB |
| Used disk space | 55 TB | 155 TB |
| Number of Files | 735 k | 737 k |
| Number of Dead files | 22 k | 232 k |
| Number of Chunks | 992 k | 1550 k |
| Metadata at chunkservers | 13 GB | 21 GB |
| Metadata at master | 48 MB | 60 MB |

| Cluster | A | B |
|---|---|---|
| Read rate (last minute) | 583 MB/s | 380 MB/s |
| Read rate (last hour) | 562 MB/s | 384 MB/s |
| Read rate (since restart) | 589 MB/s | 49 MB/s |
| Write rate (last minute) | 1 MB/s | 101 MB/s |
| Write rate (last hour) | 2 MB/s | 117 MB/s |
| Write rate (since restart) | 25 MB/s | 13 MB/s |
| Master ops (last minute) | 325 Ops/s | 533 Ops/s |
| Master ops (last hour) | 381 Ops/s | 518 Ops/s |
| Master ops (since restart) | 202 Ops/s | 347 Ops/s |

# Deployment in Google

- Many GFS clusters
- hundreds/thousands of storage nodes each
- Managing petabytes of data
- GFS is under BigTable, etc.

# Conclusion

- GFS demonstrates how to support large-scale processing workloads on commodity hardware
  - design to tolerate frequent component failures
  - optimize for huge files that are mostly appended and read
  - feel free to relax and extend FS interface as required
  - go for simple solutions (e.g., single master)
- GFS has met Google's storage needs… it must be good!

# Next Time...

- Start learning about algorithms for distributed computation
  - Clustering Algorithms